R Programming

Unit 2 - Project Assignment

Student Performance Analytics System

Subject	R Programming
Unit	Unit 2 - Data Structures and Functions
Assignment	Student Performance Analytics System
Name	Pawar Shubham Dilip
MIS No	752562026

Scenario: A college wants to analyze the performance of 120 students across five subjects using only base R. No external packages are used.

Part 1: Vectors and Basic Operations

Task 1: Generate Student IDs

```
# Task 1: Generate Student IDs from 1 to 120 using seq()
student_ids <- seq(1, 120, by = 1)

# Verify
length(student_ids)
head(student_ids, 5)
tail(student_ids, 5)
```

```
R 4.5.3 · ~/
> # Task 1: Generate Student IDs from 1 to 120 using seq()
> student_ids <- seq(1, 120, by = 1)
>
> # Verify
> length(student_ids)
[1] 120
> head(student_ids, 5)
[1] 1 2 3 4 5
> tail(student_ids, 5)
[1] 116 117 118 119 120
>
```

Task 2: Create Section Vector

```
# Task 2: Create Section vector using rep()
# 40 students each in Section A, B, C
sections <- rep(c("A", "B", "C"), each = 40)
```

```
# Verify
length(sections)
table(sections)
head(sections, 5)
sections[39:42]         (boundary check)
```

```
R 4.5.3 · ~/
> # Task 2: Create Section vector using rep()
> # 40 students each in Section A, B, C
> sections <- rep(c("A", "B", "C"), each = 40)
>
> # Verify
> length(sections)
[1] 120
> table(sections)
sections
 A  B  C
40 40 40
> head(sections, 5)
[1] "A" "A" "A" "A" "A"
> sections[39:42]
[1] "A" "A" "B" "B"
>
```

Task 3: Create Gender Vector

```
# Task 3: 60 Male + 60 Female students, randomized order
set.seed(42)  # For reproducibility
gender_base <- c(rep("Male", 60), rep("Female", 60))
gender <- sample(gender_base)  # randomize order

# Verify
length(gender)
table(gender)
head(gender, 10)
```

```
R 4.5.3 · ~/
> # Task 3: 60 Male + 60 Female students, randomized order
> set.seed(42)  # For reproducibility
> gender_base <- c(rep("Male", 60), rep("Female", 60))
> gender <- sample(gender_base)  # randomize order
>
> # Verify
> length(gender)
[1] 120
> table(gender)
gender
Female   Male
    60     60
> head(gender, 10)
 [1] "Male"   "Female" "Female" "Male"   "Female" "Female" "Male"   "Female" "Male"   "Male"
>
```

Task 4: Generate Subject Marks with NA Handling

```
# Task 4: Generate marks for all 5 subjects
# Each: random marks 40-100, 10 NA values introduced

set.seed(100)
generate_marks <- function(n = 120, na_count = 10) {
  marks <- sample(40:100, n, replace = TRUE)
  na_positions <- sample(1:n, na_count)
  marks[na_positions] <- NA
  return(marks)
}

math_marks  <- generate_marks()
stats_marks <- generate_marks()
prog_marks  <- generate_marks()
db_marks    <- generate_marks()
ml_marks    <- generate_marks()

# Count NAs in each subject
cat("NA counts - Math:", sum(is.na(math_marks)),
    "| Stats:", sum(is.na(stats_marks)),
    "| Prog:",  sum(is.na(prog_marks)),
    "| DB:",    sum(is.na(db_marks)),
    "| ML:",    sum(is.na(ml_marks)), "\n")

# Calculate subject means (na.rm removes NAs before computing mean)
means <- c(
  Math  = mean(math_marks,  na.rm = TRUE),
  Stats = mean(stats_marks, na.rm = TRUE),
  Prog  = mean(prog_marks,  na.rm = TRUE),
  DB    = mean(db_marks,    na.rm = TRUE),
  ML    = mean(ml_marks,    na.rm = TRUE)
)
cat("Subject Means:\n"); print(round(means, 2))

# Replace NA with respective subject mean
math_marks[is.na(math_marks)]   <- round(means["Math"])
stats_marks[is.na(stats_marks)] <- round(means["Stats"])
prog_marks[is.na(prog_marks)]   <- round(means["Prog"])
db_marks[is.na(db_marks)]       <- round(means["DB"])
ml_marks[is.na(ml_marks)]       <- round(means["ML"])

# Verify no NAs remain
cat("NAs after replacement:", sum(is.na(math_marks)), "\n")
```

R 4.5.3 · ~/

```
> # Task 4: Generate marks for all 5 subjects
> # Each: random marks 40-100, 10 NA values introduced
> set.seed(100)
> generate_marks <- function(n = 120, na_count = 10) {
+   marks <- sample(40:100, n, replace = TRUE)
+   na_positions <- sample(1:n, na_count)
+   marks[na_positions] <- NA
+   return(marks)
+ }
>
> math_marks  <- generate_marks()
> stats_marks <- generate_marks()
> prog_marks  <- generate_marks()
> db_marks    <- generate_marks()
> ml_marks    <- generate_marks()
>
> # Count NAs in each subject
> cat("NA counts - Math:", sum(is.na(math_marks)),
+     "| Stats:", sum(is.na(stats_marks)),
+     "| Prog:",  sum(is.na(prog_marks)),
+     "| DB:",    sum(is.na(db_marks)),
+     "| ML:",    sum(is.na(ml_marks)), "\n")
NA counts - Math: 10 | Stats: 10 | Prog: 10 | DB: 10 | ML: 10
>
> # Calculate subject means (na.rm removes NAs before computing mean)
> means <- c(
+   Math  = mean(math_marks,  na.rm = TRUE),
+   Stats = mean(stats_marks, na.rm = TRUE),
+   Prog  = mean(prog_marks,  na.rm = TRUE),
+   DB    = mean(db_marks,    na.rm = TRUE),
+   ML    = mean(ml_marks,    na.rm = TRUE)
+ )
> cat("Subject Means:\n"); print(round(means, 2))
Subject Means:
 Math Stats  Prog    DB    ML
70.01 67.05 71.23 70.05 68.80
>
> # Replace NA with respective subject mean
> math_marks[is.na(math_marks)]   <- round(means["Math"])
> stats_marks[is.na(stats_marks)] <- round(means["Stats"])
> prog_marks[is.na(prog_marks)]   <- round(means["Prog"])
> db_marks[is.na(db_marks)]       <- round(means["DB"])
```

```
>
> # Verify no NAs remain
> cat("NAs after replacement:", sum(is.na(math_marks)), "\n")  # 0
NAs after replacement: 0
>
```

Part 2: Matrices

Task 5: Create Marks Matrix

```
# Task 5: Combine all 5 subject vectors into a matrix
# Rows = students (120), Columns = subjects (5)
marks_matrix <- cbind(math_marks, stats_marks, prog_marks, db_marks, ml_marks)
colnames(marks_matrix) <- c("Mathematics", "Statistics", "Programming", "Database",
"MachineLearning")
rownames(marks_matrix) <- paste0("S", student_ids)

cat("Matrix dimensions:", nrow(marks_matrix), "rows x", ncol(marks_matrix),
"cols\n")
# Output: Matrix dimensions: 120 rows x 5 cols

# 1. Extract marks of Student 10
cat("\nStudent 10 marks:\n")
print(marks_matrix[10, ])

# 2. Extract all Programming marks
cat("\nFirst 10 Programming marks:\n")
print(head(marks_matrix[ , "Programming"], 10))

# 3. Add 5 grace marks to entire matrix
marks_matrix_graced <- marks_matrix + 5
# Cap at 100 (marks can't exceed 100)
marks_matrix_graced[marks_matrix_graced > 100] <- 100
cat("\nGrace marks added (capped at 100). Student 1 before/after:\n")
cat("Before:", marks_matrix[1, ], "\n")
cat("After :", marks_matrix_graced[1, ], "\n")

# 4. Total marks per student using rowSums()
total_marks <- rowSums(marks_matrix)
cat("\nTotal marks - first 5 students:\n")
print(head(total_marks, 5))

# 5. Subject averages using colMeans()
subject_averages <- colMeans(marks_matrix)
cat("\nSubject-wise Averages:\n")
print(round(subject_averages, 2))
```

R 4.5.3 · ~/

```
> # Task 5: Combine all 5 subject vectors into a matrix
> # Rows = students (120), Columns = subjects (5)
> marks_matrix <- cbind(math_marks, stats_marks, prog_marks, db_marks, ml_marks)
> colnames(marks_matrix) <- c("Mathematics", "Statistics", "Programming", "Database", "MachineLearnin
g")
> rownames(marks_matrix) <- paste0("S", student_ids)
>
> cat("Matrix dimensions:", nrow(marks_matrix), "rows x", ncol(marks_matrix), "cols\n")
Matrix dimensions: 120 rows x 5 cols
> # Output: Matrix dimensions: 120 rows x 5 cols
>
> # 1. Extract marks of Student 10
> cat("\nStudent 10 marks:\n")

Student 10 marks:
> print(marks_matrix[10, ])
    Mathematics      Statistics     Programming        Database MachineLearning
             62              62              59              79              58
>
> # 2. Extract all Programming marks
> cat("\nFirst 10 Programming marks:\n")

First 10 Programming marks:
> print(head(marks_matrix[ , "Programming"], 10))
 S1  S2  S3  S4  S5  S6  S7  S8  S9 S10
 71  71  44  59  83  58  85  91  91  59
>
> # 3. Add 5 grace marks to entire matrix
> marks_matrix_graced <- marks_matrix + 5
> # Cap at 100 (marks can't exceed 100)
> marks_matrix_graced[marks_matrix_graced > 100] <- 100
> cat("\nGrace marks added (capped at 100). Student 1 before/after:\n")

Grace marks added (capped at 100). Student 1 before/after:
> cat("Before:", marks_matrix[1, ], "\n")
Before: 49 69 71 44 40
> cat("After :", marks_matrix_graced[1, ], "\n")
After : 54 74 76 49 45
>
> # 4. Total marks per student using rowSums()
> total_marks <- rowSums(marks_matrix)
> cat("\nTotal marks - first 5 students:\n")
```

```
Total marks - first 5 students:
> print(head(total_marks, 5))
 S1  S2  S3  S4  S5
273 389 306 309 388
>
> # 5. Subject averages using colMeans()
> subject_averages <- colMeans(marks_matrix)
> cat("\nSubject-wise Averages:\n")

Subject-wise Averages:
> print(round(subject_averages, 2))
    Mathematics      Statistics     Programming        Database MachineLearning
          70.01           67.04           71.21           70.04           68.82
>
```

Part 3: Lists

Task 6: Create a Student Record List

```
# Task 6: Create a comprehensive list for student records
percentage <- round((total_marks / 500) * 100, 2)

student_record <- list(
  StudentIDs   = student_ids,
  Section      = sections,
  Gender       = gender,
  MarksMatrix  = marks_matrix,
  TotalMarks   = total_marks
)

# Access individual elements
cat("Number of students:", length(student_record$StudentIDs), "\n")
cat("First student ID  :", student_record$StudentIDs[1], "\n")
cat("First student total marks:", student_record$TotalMarks[1], "\n")

# Add a new element: Percentage
student_record$Percentage <- percentage
cat("\nElement added: Percentage\n")
cat("First student percentage:", student_record$Percentage[1], "%\n")

# Remove one element using NULL
student_record$MarksMatrix <- NULL
cat("MarksMatrix removed. Remaining elements:\n")
cat(names(student_record), "\n")

# Check structure
str(student_record)
```

R 4.5.3 · ~/

```
> # Task 6: Create a comprehensive list for student records
> percentage <- round((total_marks / 500) * 100, 2)
>
> student_record <- list(
+   StudentIDs    = student_ids,
+   Section       = sections,
+   Gender        = gender,
+   MarksMatrix   = marks_matrix,
+   TotalMarks    = total_marks
+ )
>
> # Access individual elements
> cat("Number of students:", length(student_record$StudentIDs), "\n")
Number of students: 120
> cat("First student ID  :", student_record$StudentIDs[1], "\n")
First student ID  : 1
> cat("First student total marks:", student_record$TotalMarks[1], "\n")
First student total marks: 273
>
> # Add a new element: Percentage
> student_record$Percentage <- percentage
> cat("\nElement added: Percentage\n")

Element added: Percentage
> cat("First student percentage:", student_record$Percentage[1], "%\n")
First student percentage: 54.6 %
>
> # Remove one element using NULL
> student_record$MarksMatrix <- NULL
> cat("MarksMatrix removed. Remaining elements:\n")
MarksMatrix removed. Remaining elements:
> cat(names(student_record), "\n")
StudentIDs Section Gender TotalMarks Percentage
>
> # Check structure
> str(student_record)
List of 5
 $ StudentIDs: num [1:120] 1 2 3 4 5 6 7 8 9 10 ...
 $ Section   : chr [1:120] "A" "A" "A" "A" ...
 $ Gender    : chr [1:120] "Male" "Female" "Female" "Male" ...
 $ TotalMarks: Named num [1:120] 273 389 306 309 388 306 426 309 373 320 ...
  ..- attr(*, "names")= chr [1:120] "S1" "S2" "S3" "S4" ...
```

```
 $ Percentage: Named num [1:120] 54.6 77.8 61.2 61.8 77.6 61.2 85.2 61.8 74.6 64 ...
  ..- attr(*, "names")= chr [1:120] "S1" "S2" "S3" "S4" ...
>
```

Search

Part 4: Data Frames

Task 7: Create Data Frame

```
# Task 7: Build the full data frame
percentage <- round((total_marks / 500) * 100, 2)

student_df <- data.frame(
  StudentID    = student_ids,
  Section      = sections,
  Gender       = gender,
  Mathematics  = math_marks,
  Statistics   = stats_marks,
  Programming  = prog_marks,
  Database     = db_marks,
  MachineLrn   = ml_marks,
  Total        = total_marks,
  Percentage   = percentage
)

cat("Data frame created:", nrow(student_df), "rows x", ncol(student_df), "cols\n")
head(student_df, 3)

# 1. Extract female students
female_students <- student_df[student_df$Gender == "Female", ]
cat("\nFemale students count:", nrow(female_students), "\n")

# 2. Extract Section B students
section_b <- student_df[student_df$Section == "B", ]
cat("Section B students count:", nrow(section_b), "\n")

# 3. Extract students with Percentage > 75
high_pct <- student_df[student_df$Percentage > 75, ]
cat("Students with % > 75:", nrow(high_pct), "\n")

# 4. Select only ID and Percentage columns
id_pct <- student_df[ , c("StudentID", "Percentage")]
cat("\nID and Percentage (first 5):\n")
print(head(id_pct, 5))
```

R 4.5.3 · ~/

```
> # Task 7: Build the full data frame
> percentage <- round((total_marks / 500) * 100, 2)
>
> student_df <- data.frame(
+   StudentID    = student_ids,
+   Section      = sections,
+   Gender       = gender,
+   Mathematics  = math_marks,
+   Statistics   = stats_marks,
+   Programming  = prog_marks,
+   Database     = db_marks,
+   MachineLrn   = ml_marks,
+   Total        = total_marks,
+   Percentage   = percentage
+ )
>
> cat("Data frame created:", nrow(student_df), "rows x", ncol(student_df), "cols\n")
Data frame created: 120 rows x 10 cols
> head(student_df, 3)
   StudentID Section Gender Mathematics Statistics Programming Database MachineLrn Total
S1         1       A   Male          49         69          71       44         40   273
S2         2       A Female          94         66          71       79         79   389
S3         3       A Female          77         57          44       63         65   306
   Percentage
S1       54.6
S2       77.8
S3       61.2
>
> # 1. Extract female students
> female_students <- student_df[student_df$Gender == "Female", ]
> cat("\nFemale students count:", nrow(female_students), "\n")

Female students count: 60
>
> # 2. Extract Section B students
> section_b <- student_df[student_df$Section == "B", ]
> cat("Section B students count:", nrow(section_b), "\n")
Section B students count: 40
>
> # 3. Extract students with Percentage > 75
> high_pct <- student_df[student_df$Percentage > 75, ]
```

```
> cat("Students with % > 75:", nrow(high_pct), "\n")
Students with % > 75: 30
>
> # 4. Select only ID and Percentage columns
> id_pct <- student_df[ , c("StudentID", "Percentage")]
> cat("\nID and Percentage (first 5):\n")

ID and Percentage (first 5):
> print(head(id_pct, 5))
   StudentID Percentage
S1         1       54.6
S2         2       77.8
S3         3       61.2
S4         4       61.8
S5         5       77.6
>
```

Part 5: rbind() and cbind()

Task 8

```
# Task 8.1: Add 5 new students using rbind()
new_students <- data.frame(
  StudentID   = 121:125,
  Section     = c("A", "B", "C", "A", "B"),
  Gender      = c("Male", "Female", "Male", "Female", "Male"),
  Mathematics = c(88, 72, 90, 65, 78),
  Statistics  = c(85, 68, 88, 70, 74),
  Programming = c(80, 75, 92, 60, 82),
  Database    = c(90, 70, 85, 72, 79),
  MachineLrn  = c(86, 66, 91, 68, 77),
  Total       = c(429, 351, 446, 335, 390),
  Percentage  = c(85.8, 70.2, 89.2, 67.0, 78.0)
)

student_df_extended <- rbind(student_df, new_students)
cat("After rbind() - Total rows:", nrow(student_df_extended), "\n")
# Output: After rbind() - Total rows: 125

# Task 8.2: Add Attendance column using cbind()
set.seed(55)
attendance_vec <- sample(60:100, nrow(student_df_extended), replace = TRUE)
student_df_extended <- cbind(student_df_extended, Attendance = attendance_vec)
cat("After cbind() - Total columns:", ncol(student_df_extended), "\n")

# Task 8.3: Alternative method using data.frame()
student_df_alt <- data.frame(student_df_extended,
  Remarks = ifelse(student_df_extended$Percentage >= 75, "Pass", "Needs
Improvement")
)
cat("Alternative method - columns:", colnames(student_df_alt), "\n")
```

```
> # Task 8.1: Add 5 new students using rbind()
> new_students <- data.frame(
+   StudentID   = 121:125,
+   Section     = c("A", "B", "C", "A", "B"),
+   Gender      = c("Male", "Female", "Male", "Female", "Male"),
+   Mathematics = c(88, 72, 90, 65, 78),
+   Statistics  = c(85, 68, 88, 70, 74),
+   Programming = c(80, 75, 92, 60, 82),
+   Database    = c(90, 70, 85, 72, 79),
+   MachineLrn  = c(86, 66, 91, 68, 77),
+   Total       = c(429, 351, 446, 335, 390),
+   Percentage  = c(85.8, 70.2, 89.2, 67.0, 78.0)
+ )
>
> student_df_extended <- rbind(student_df, new_students)
> cat("After rbind() - Total rows:", nrow(student_df_extended), "\n")
After rbind() - Total rows: 125
> # Output: After rbind() - Total rows: 125
>
> # Task 8.2: Add Attendance column using cbind()
> set.seed(55)
> attendance_vec <- sample(60:100, nrow(student_df_extended), replace = TRUE)
> student_df_extended <- cbind(student_df_extended, Attendance = attendance_vec)
> cat("After cbind() - Total columns:", ncol(student_df_extended), "\n")
After cbind() - Total columns: 11
>
> # Task 8.3: Alternative method using data.frame()
> student_df_alt <- data.frame(student_df_extended,
+                              Remarks = ifelse(student_df_extended$Percentage >= 75, "Pass", "Needs Im
provement")
+ )
> cat("Alternative method - columns:", colnames(student_df_alt), "\n")
Alternative method - columns: StudentID Section Gender Mathematics Statistics Programming Database Mach
ineLrn Total Percentage Attendance Remarks
>
```

Part 6: Factors

Task 9: Convert to Factors

```
# Task 9: Convert Section and Gender to factors
student_df$Section <- factor(student_df$Section)
student_df$Gender  <- factor(student_df$Gender)

# Check levels
cat("Section levels:", levels(student_df$Section), "\n")
# Output: Section levels: A B C
cat("Gender levels :", levels(student_df$Gender), "\n")
# Output: Gender levels : Female Male

# Rename factor levels (optional relabeling)
levels(student_df$Section) <- c("Section-A", "Section-B", "Section-C")
cat("Renamed Section levels:", levels(student_df$Section), "\n")

# Reset to original labels for further analysis
levels(student_df$Section) <- c("A", "B", "C")

# Count students per section
cat("\nStudents per section:\n")
print(table(student_df$Section))
```

```
R 4.5.3 · ~/
> # Task 9: Convert Section and Gender to factors
> student_df$Section <- factor(student_df$Section)
> student_df$Gender  <- factor(student_df$Gender)
>
> # Check levels
> cat("Section levels:", levels(student_df$Section), "\n")
Section levels: A B C
> # Output: Section levels: A B C
> cat("Gender levels :", levels(student_df$Gender), "\n")
Gender levels : Female Male
> # Output: Gender levels : Female Male
>
> # Rename factor levels (optional relabeling)
> levels(student_df$Section) <- c("Section-A", "Section-B", "Section-C")
> cat("Renamed Section levels:", levels(student_df$Section), "\n")
Renamed Section levels: Section-A Section-B Section-C
>
> # Reset to original labels for further analysis
> levels(student_df$Section) <- c("A", "B", "C")
>
> # Count students per section
> cat("\nStudents per section:\n")

Students per section:
> print(table(student_df$Section))

 A  B  C
40 40 40
>
```

Task 10: Create Grade Factor

```
# Task 10: Create ordered Grade factor based on Percentage
student_df$Grade <- cut(
  student_df$Percentage,
  breaks = c(-Inf, 50, 70, 85, Inf),
  labels = c("D", "C", "B", "A"),
  right  = TRUE
)

# Convert to ordered factor
student_df$Grade <- factor(student_df$Grade,
  levels  = c("D", "C", "B", "A"),
  ordered = TRUE
)

# Verify ordered factor
is.ordered(student_df$Grade)  # TRUE
cat("Grade levels (ordered):", levels(student_df$Grade), "\n")
# Output: Grade levels (ordered): D C B A

# Grade frequency distribution
cat("\nGrade Distribution:\n")
print(table(student_df$Grade))

# Proportion table
cat("\nGrade Proportions:\n")
print(round(prop.table(table(student_df$Grade)) * 100, 1))
```

```
R 4.5.3 · ~/
> # Task 10: Create ordered Grade factor based on Percentage
> student_df$Grade <- cut(
+   student_df$Percentage,
+   breaks = c(-Inf, 50, 70, 85, Inf),
+   labels = c("D", "C", "B", "A"),
+   right  = TRUE
+ )
>
> # Convert to ordered factor
> student_df$Grade <- factor(student_df$Grade,
+                            levels  = c("D", "C", "B", "A"),
+                            ordered = TRUE
+ )
>
> # Verify ordered factor
> is.ordered(student_df$Grade)  # TRUE
[1] TRUE
> cat("Grade levels (ordered):", levels(student_df$Grade), "\n")
Grade levels (ordered): D C B A
> # Output: Grade levels (ordered): D C B A
>
> # Grade frequency distribution
> cat("\nGrade Distribution:\n")

Grade Distribution:
> print(table(student_df$Grade))

 D  C  B  A
 0 66 52  2
>
> # Proportion table
> cat("\nGrade Proportions:\n")

Grade Proportions:
> print(round(prop.table(table(student_df$Grade)) * 100, 1))

   D    C    B    A
 0.0 55.0 43.3  1.7
>
```

Part 7: split() and by()

Task 11: Split Data by Section

```
# Task 11: Split data by Section
section_split <- split(student_df, student_df$Section)

# Calculate mean percentage of each section
cat("=== Section-wise Mean Percentage ===\n")
for (sec in names(section_split)) {
  mean_pct <- mean(section_split[[sec]]$Percentage)
  cat("Section", sec, ": Mean %", round(mean_pct, 2), "\n")
}

# More concise with sapply:
section_means <- sapply(section_split, function(s) round(mean(s$Percentage), 2))
cat("\nSapply result:\n")
print(section_means)
```

```
cat("Best performing section:", names(which.max(section_means)), "\n")
```

```
R 4.5.3 · ~/
> # Task 11: Split data by Section
> section_split <- split(student_df, student_df$Section)
>
> # Calculate mean percentage of each section
> cat("=== Section-wise Mean Percentage ===\n")
=== Section-wise Mean Percentage ===
> for (sec in names(section_split)) {
+   mean_pct <- mean(section_split[[sec]]$Percentage)
+   cat("Section", sec, ": Mean %", round(mean_pct, 2), "\n")
+ }
Section A : Mean % 69.81
Section B : Mean % 68.61
Section C : Mean % 69.86
>
> # More concise with sapply:
> section_means <- sapply(section_split, function(s) round(mean(s$Percentage), 2))
> cat("\nSapply result:\n")

Sapply result:
> print(section_means)
    A     B     C
69.81 68.61 69.86
> cat("Best performing section:", names(which.max(section_means)), "\n")
Best performing section: C
>
```

Task 12: by() for Group Analysis

```
# Task 12: Use by() to calculate averages per gender and per section

# Average marks per gender
cat("=== Average Marks per Gender ===\n")
gender_avgs <- by(student_df$Percentage, student_df$Gender, mean)
print(round(gender_avgs, 2))

# Average marks per section
cat("\n=== Average Marks per Section ===\n")
section_avgs <- by(student_df$Percentage, student_df$Section, mean)
print(round(section_avgs, 2))

# by() for multiple statistics per group (use a function)
cat("\n=== Detailed Stats per Gender ===\n")
by(student_df$Percentage, student_df$Gender, function(x) {
  c(Mean = round(mean(x), 2),
    Median = round(median(x), 2),
    SD = round(sd(x), 2),
    Min = min(x), Max = max(x))
})
```

R 4.5.3 · ~/

```
> # Task 12: Use by() to calculate averages per gender and per section
>
> # Average marks per gender
> cat("=== Average Marks per Gender ===\n")
=== Average Marks per Gender ===
> gender_avgs <- by(student_df$Percentage, student_df$Gender, mean)
> print(round(gender_avgs, 2))
student_df$Gender: Female
[1] 68.69
------------------------------------------------------------------------
student_df$Gender: Male
[1] 70.16
>
> # Average marks per section
> cat("\n=== Average Marks per Section ===\n")

=== Average Marks per Section ===
> section_avgs <- by(student_df$Percentage, student_df$Section, mean)
> print(round(section_avgs, 2))
student_df$Section: A
[1] 69.81
------------------------------------------------------------------------
student_df$Section: B
[1] 68.61
------------------------------------------------------------------------
student_df$Section: C
[1] 69.86
>
> # by() for multiple statistics per group (use a function)
> cat("\n=== Detailed Stats per Gender ===\n")

=== Detailed Stats per Gender ===
> by(student_df$Percentage, student_df$Gender, function(x) {
+   c(Mean = round(mean(x), 2),
+     Median = round(median(x), 2),
+     SD = round(sd(x), 2),
+     Min = min(x), Max = max(x))
+ })
student_df$Gender: Female
  Mean Median     SD    Min    Max
 68.69  68.30   7.72  54.00  84.60
------------------------------------------------------------------------
student_df$Gender: Male
  Mean Median     SD    Min    Max
 70.16  69.60   8.31  52.20  89.80
>
```

Final Deliverable: Summary Report

Subject-wise Average Marks

```
cat("=====================================\n")
cat("     STUDENT PERFORMANCE REPORT       \n")
cat("=====================================\n")

cat("\n--- Subject-wise Averages ---\n")
print(round(colMeans(marks_matrix), 2))

cat("\n--- Section-wise Performance ---\n")
print(round(by(student_df$Percentage, student_df$Section, mean), 2))

cat("\n--- Gender-wise Analysis ---\n")
print(round(by(student_df$Percentage, student_df$Gender, mean), 2))

cat("\n--- Grade Distribution ---\n")
print(table(student_df$Grade))

cat("\n--- Top 5 Students ---\n")
top5 <- head(student_df[order(-student_df$Percentage),
              c("StudentID", "Section", "Gender", "Total", "Percentage", "Grade")],
5)
print(top5)

cat("\n=====================================\n")
```

```
R 4.5.3 · ~/
> cat("======================================\n")
======================================
> cat("     STUDENT PERFORMANCE REPORT       \n")
     STUDENT PERFORMANCE REPORT
> cat("======================================\n")
======================================
>
> cat("\n--- Subject-wise Averages ---\n")

--- Subject-wise Averages ---
> print(round(colMeans(marks_matrix), 2))
    Mathematics      Statistics     Programming        Database MachineLearning
          70.01           67.04           71.21           70.04           68.82
>
> cat("\n--- Section-wise Performance ---\n")

--- Section-wise Performance ---
> print(round(by(student_df$Percentage, student_df$Section, mean), 2))
student_df$Section: A
[1] 69.81
------------------------------------------------------------------------
student_df$Section: B
[1] 68.61
------------------------------------------------------------------------
student_df$Section: C
[1] 69.86
>
> cat("\n--- Gender-wise Analysis ---\n")

--- Gender-wise Analysis ---
> print(round(by(student_df$Percentage, student_df$Gender, mean), 2))
student_df$Gender: Female
[1] 68.69
------------------------------------------------------------------------
student_df$Gender: Male
[1] 70.16
>
> cat("\n--- Grade Distribution ---\n")

--- Grade Distribution ---
> print(table(student_df$Grade))
```

```
 D  C  B  A
 0 66 52  2
>
> cat("\n--- Top 5 Students ---\n")

--- Top 5 Students ---
> top5 <- head(student_df[order(-student_df$Percentage),
+                         c("StudentID", "Section", "Gender", "Total", "Percentage", "Grade")], 5)
> print(top5)
    StudentID Section Gender Total Percentage Grade
S79        79       B   Male   449       89.8     A
S7          7       A   Male   426       85.2     A
S39        39       A Female   423       84.6     B
S53        53       B   Male   419       83.8     B
S31        31       A   Male   414       82.8     B
>
> cat("\n======================================\n")

======================================
> |
```

End of Assignment

R Programming - Unit 2 | Vectors, Matrices, Lists, Data Frames, Factors, split() and by()